Análise e Desenvolvimento de Sistemas

Disciplina: CODING

Professora: Diego Abreu

Aluno: Giovani Amauri Frazão Brelaz

Matrícula: 04206364

Turno: Manhã

Aula 5 Parte 2 Decomposição Escopo e Passagem por Valor.docx

Atividade 1 - Sistema acadêmico decomposto

Leia três notas, calcule a média e informe a situação. Em vez de colocar toda a lógica dentro de main, use as funções abaixo.

```
double calcularMedia(double a, double b, double c) {
return (a + b + c) / 3.0;
}

void mostrarSituacao(double media) {
if (media >= 7.0) {
printf("Aprovado\n");
} else {
printf("Reprovado\n");
}
}
```

No main, faça apenas a entrada das notas, a chamada de calcularMedia(), a impressão da média e a chamada de mostrarSituacao().

```c
#include <stdio.h>

double calcularMedia(double a, double b, double c) {
    return (a + b + c) / 3.0;
}

void mostrarSituacao(double media) {
    if (media >= 7.0) {
        printf("Aprovado\n");
    } else {
        printf("Reprovado\n");
    }
}

int main(void) {
    double n1, n2, n3, media;
    printf("Digite a nota 1: ");
    scanf("%lf", &n1);
    printf("Digite a nota 2: ");
    scanf("%lf", &n2);
    printf("Digite a nota 3: ");
    scanf("%lf", &n3);

    media = calcularMedia(n1, n2, n3);

    printf("Media = %.2f\n", media);
    mostrarSituacao(media);
    return 0;
}
```

Pergunta de projeto

Qual responsabilidade pertence a cada função?

- calcularMedia(a, b, c) -> Só calcula.

- mostrarSituacao(media) -> Só decide e mostra.

- main -> Só coordena.

Há alguma responsabilidade que ainda está diretamente em main?

Sim. Ainda ficou no main:

- A leitura das notas com scanf

- A impressão da média com printf

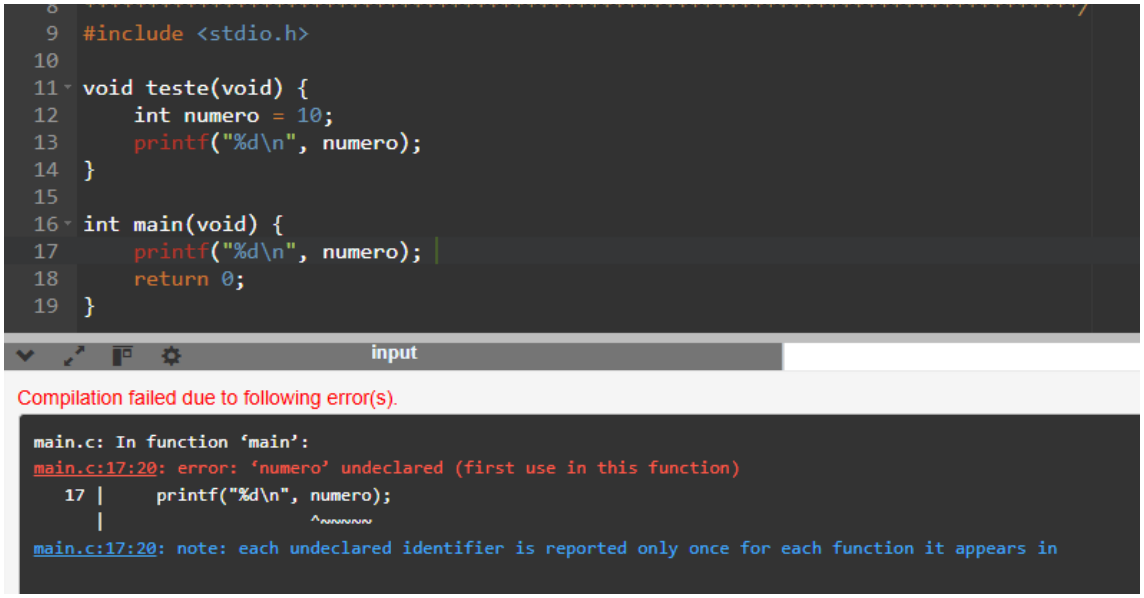
Atividade 2 - Escopo local

```
void teste(void) {
int numero = 10;
printf("%d\n", numero);
}

 int main(void) {
printf("%d\n", numero);
return 0;
}
```

Tente compilar. O segundo printf não consegue acessar numero, pois numero é local à função teste().

Regra prática: mantenha os dados no menor escopo necessário.



Atividade 3 - Escopo de bloco

int idade = 20;

if (idade >= 18) {
int desconto = 10;
printf("Desconto: %d\n", desconto);
}

/* tente usar desconto aqui */



Teste também um for com int i declarado dentro do próprio cabeçalho e tente usar i após o laço.



```c
#include <stdio.h>

int main(void) {
    int i;
    for (i = 0; i < 3; i++) {
        printf("%d\n", i);
    }
    printf("Valor final de i: %d\n", i);
    return 0;
}
```

input

```
0
1
2
Valor final de i: 3


...Program finished with exit code 0
Press ENTER to exit console.
```

Atividade 4 - Local versus global

Compare as duas ideias:

```
int calcularDobro(int numero) {
return numero * 2;
}
```

```
int numeroGlobal;

int calcularDobro(void) {
return numeroGlobal * 2;
}
```

1. Qual versão deixa mais explícito de onde vem o valor usado no cálculo?

A Versão 1, com parâmetro.
Quando você olha calcularDobro(10), você já sabe na hora que o valor é 10. Está escrito na chamada.

A versão 2, com global.
Quando você olha calcularDobro(), você fica se perguntando de onde veio aquele valor. Você tem que caçar no programa inteiro para descobrir.

2. Qual versão depende de um estado externo à função?

A Versão 2, com global.

Ela depende de uma variável que está fora dela.

Atenção: variáveis globais existem e podem ser úteis, mas para este curso prefira parâmetros, retornos e variáveis locais sempre que possível.

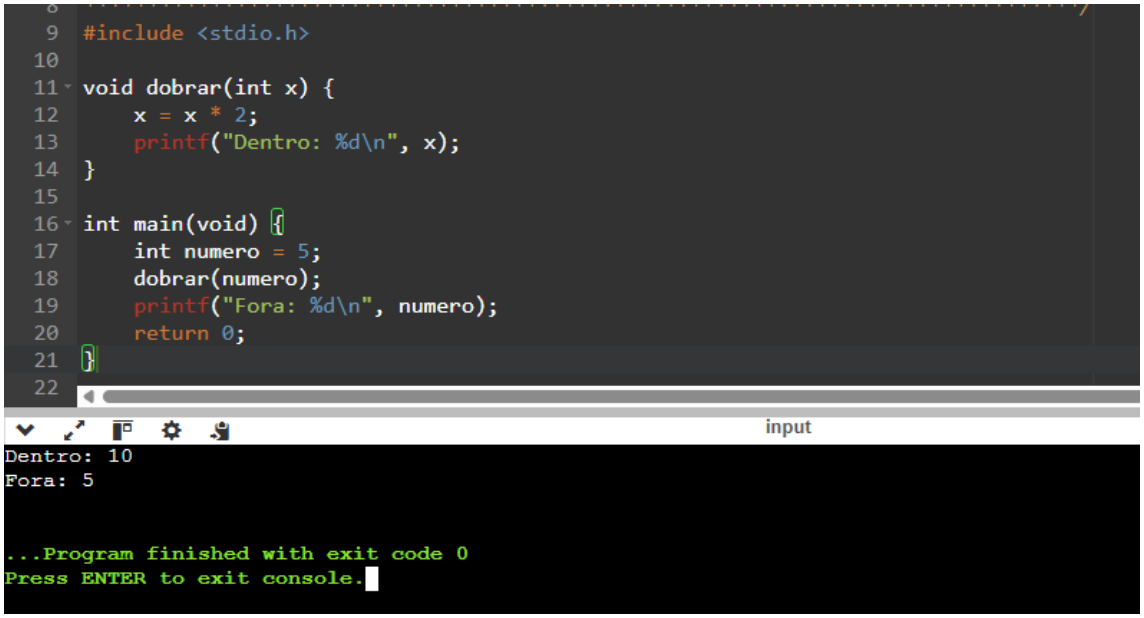
Atividade 5 - Passagem por valor

Preveja o resultado antes de executar.

```
void dobrar(int x) {
x = x * 2;
printf("Dentro: %d\n", x);
}

int main(void) {
int numero = 5;
dobrar(numero);
printf("Fora: %d\n", numero);
return 0;
}
```

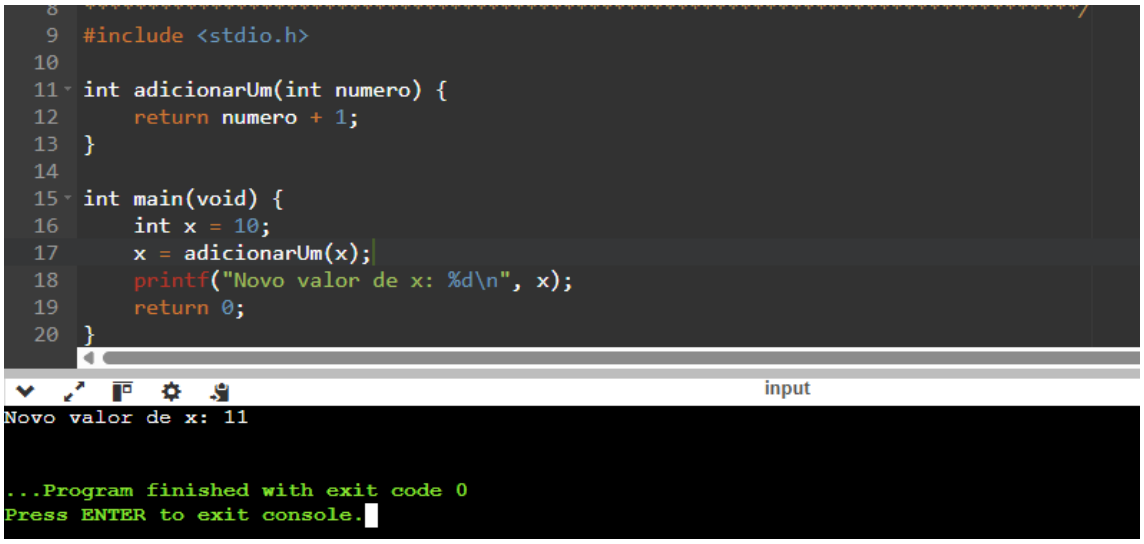
Saída esperada: dentro da função x vale 10, mas número continua 5. A função recebeu uma cópia do valor.



Desafio - Corrigindo a intenção

A função abaixo não altera x em main. Reescreva o programa para que o novo valor seja obtido por retorno.

```
void adicionarUm(int numero) {
numero++;
}
```



3. Por que uma variável local de uma função não pode ser usada automaticamente em outra?

Resposta: Porque ela só existe dentro da função onde foi criada. Cada função tem seu próprio espaço, ela não enxerga as variáveis locais das outras.

4. Qual é a diferença entre escopo de função e escopo de bloco?

Resposta: Escopo de Função: É a variável que vale para a função inteira, do início ao fim dela.

Escopo de Bloco: É menor ainda, é a variável que só vale dentro de um { } específico, como dentro de um if ou de um for.

5. Por que número continua 5 após a chamada de dobrar(numero)?

Resposta: A função recebe só uma cópia do número, não o original. Ela dobra a cópia para 10 dentro dela, mas o número original no main continua 5.